

PyLCF — Anleitung

Grafische Oberfläche zur Linearkombinations-Anpassung (LCF) von Spektren und anderen 1D-Daten Version 1.0.0 · Programm: `pylcf` (Start: `python -m pylcf`)

1. Worum geht es?

LCF steht für *Linear Combination Fitting* — auf Deutsch *Linearkombinations-Fit*. **PyLCF** ist die zugehörige grafische Anwendung.

Ein gemessener Datensatz — eine Kurve $y(x)$ über einer Achse x (Energie, 2θ , Zeit, Wellenlänge, m/z ...) — wird als **gewichtete Summe** von Referenz-Datensätzen auf derselben Achse beschrieben:

$$y_{\text{gemessen}}(x) \approx \sum_i w_i \cdot y_i(x)$$

Das Programm sucht die Gewichte w_i . Bei flächennormierten Daten und im Modus *convex* sind diese Gewichte direkt als **Anteile** lesbar (alle $w_i \geq 0$, Summe = 1) — z. B. Populationsanteile.

Durchgängiges Beispiel in dieser Anleitung ist ein **NIS/NRVS-Spektrum**, das in seine Komponenten-PVDOS zerlegt wird (Fe-PVDOS verschiedener Spezies oder Protonierungszustände aus DFT). Das Verfahren ist aber **nicht auf Spektren beschränkt**: Es passt für alle Daten, die plausibel eine **Linearkombination** von Referenzen auf einer gemeinsamen x -Achse sind — etwa XAS/XANES-LCF, Beugungsmuster, Chromatogramme oder kinetische Verläufe.

Was es nicht ist: kein allgemeiner (nichtlinearer) Kurven-Fit und keine Peak-Anpassung. Die Referenzen werden nur in ihrer **Amplitude** skaliert (nicht entlang x verschoben oder verzerrt); sie sollten auf vergleichbaren x -Bereichen vorliegen (unterschiedliche Raster sind erlaubt und werden interpoliert).

GUI und Kommandozeile (`pylcf-cli`) teilen sich denselben numerischen Kern (`pylcf/core.py`); die Ergebnisse sind damit per Konstruktion identisch. Die grafische Oberfläche ändert nur die *Bedienung* — vor allem die Dateneingabe. Ein Kontrolllauf mit den Testdaten liefert denselben R-Faktor ($1,614 \cdot 10^{-3}$) wie das ursprüngliche Skript.

2. Installation & Start

Voraussetzungen

Paket	Zweck	Pflicht?
Python $\geq$ 3.9	Laufzeitumgebung	ja
<code>numpy</code> , <code>scipy</code> , <code>matplotlib</code>	Rechnen & Plot	ja
<code>tkinter</code>	grafische Oberfläche	ja (bei Windows/macOS im Standard-Python enthalten)
<code>pandas</code> , <code>openpyxl</code>	nur für den Excel-Export (.xlsx)	optional

Installation der Pakete (in PowerShell):

```
pip install numpy scipy matplotlib pandas openpyxl
```

Unter Linux fehlt Tkinter manchmal und wird separat installiert (`sudo apt install python3-tk`). Unter Windows ist das nicht nötig.

Programm starten

Direkt aus dem Repository (ohne Installation):

```
python -m pylcf
```

Nach `pip install .` steht die GUI als Befehl `pylcf` bereit, eine Kommandozeile als `pylcf-cli` (bzw. `python -m pylcf.cli`). Fehlt Tkinter oder Matplotlib, gibt das Programm beim Start einen Hinweis aus und beendet sich.

Bedienspalte scrollen. Die linke Spalte (Abschnitte 1–4) kann höher sein als das Fenster — z. B. auf kleinen Bildschirmen oder bei vielen Referenzen. Sie hat deshalb rechts einen **Scrollbar**; mit ihm (oder dem **Mausrad** über der Spalte) erreichen Sie alle Schaltflächen, auch »Interactive overlay« und »Show guide« ganz unten in Abschnitt 4. Das Fenster ist außerdem frei in der Größe veränderbar.

3. Daten eingeben

Es gibt vier Wege. **Variante A (Einfügen aus Excel)** und **Variante B (Excel-Datei importieren)** sind am bequemsten. Liegen alle Spektren auf **demselben** Energieraster, eignen sich das Einfügen (A) oder der Excel-Import im Modus »Shared grid« (B). Haben die Spektren **unterschiedliche x/y-Werte** (eigenes Raster je Spektrum), nutzt man den Excel-Import im Modus »XY pairs« (B), den **Ordner-Import** (C, eine Datei je Spektrum) oder das Laden einzelner Dateien (D) — in all diesen Fällen wird beim Fit automatisch auf den gemeinsamen Überlappungsbereich interpoliert.

Variante A — Tabelle aus Excel einfügen

1. In Excel die Spalten markieren — gern mit Kopfzeile, z. B.

Energie	Gemessen	WT	DM	Red
0	0,000	0,000	0,000	0,000
2	0,012	0,015	0,010	0,008
...	...	...	...	...

2. **Strg + C** drücken.
3. Im Programm »**Paste table ...**« → »**Paste from clipboard**«.
4. Auf »**Detect columns**« klicken. Das Programm zeigt eine Vorschau und ordnet jeder Spalte eine **Rolle** zu (Auswahlmenü):
 - **Energy** — die Energieachse (cm^{-1}). Genau **eine** Spalte.
 - **Measured** — das zu beschreibende Spektrum. **Höchstens eine** Spalte.
 - **Reference** — eine Komponente/Sub-Spektrum. **Beliebig viele**.
 - **Ignore** — Spalte wird nicht verwendet.

Die **Namen** der Spalten lassen sich hier noch bearbeiten.

5. »**Apply**«. Die Spektren erscheinen in der Liste unter »1 - Data«.

Dezimalkommata und Trennzeichen werden automatisch erkannt: Beim Kopieren aus Excel sind die Spalten tab-getrennt, und ein Komma wird als Dezimaltrennzeichen interpretiert (deutsches Excel). Falls die Erkennung scheitert, lässt sie sich im Dialog über die Felder »**Delimiter**« (Tab / Semikolon / Komma / Leerzeichen) und »**Decimal**« (Punkt / Komma) erzwingen.

Hinweis: Beim Einfügen teilen sich alle Spalten **dieselbe** Energieachse. Für Spektren mit *unterschiedlichen* Energierastern eignen sich Variante B (»XY pairs«) oder Variante C.

Variante B — Excel-Datei importieren (.xlsx/.xlsm)

Statt zu kopieren lässt sich eine Excel-Datei direkt einlesen — auch mit mehreren Tabellenblättern.

1. »**Excel file ...**« klicken und die **.xlsx / .xlsm**-Datei wählen.
2. Im Dialog das **Tabellenblatt** und das **Layout** wählen:
 - **Gemeinsames Raster** — eine **Energy**-Spalte + beliebig viele Intensitätsspalten (**Measured / Reference**), genau wie bei Variante A. Alle Spektren teilen sich ein Raster.
 - **XY-Paare** — aufeinanderfolgende Spaltenpaare **E, I, E, I, ...**. Jedes Paar ist ein **eigenes Spektrum mit eigenem Energieraster**, d. h. die Spektren dürfen **unterschiedliche x/y-Werte** und sogar unterschiedlich viele Punkte haben (kürzere einfach mit leeren Zellen auffüllen). Eine einzelne überzählige Spalte ohne Partner wird ignoriert.
3. Jeder Spalte bzw. jedem Paar eine **Rolle** geben (Measured / Reference / Ignore). Enthält ein Spaltenname „gemessen“/„measured“, wird dieses Paar automatisch als **Measured** vorgeschlagen. Namen sind editierbar.
4. »**Apply**« — die Spektren erscheinen in der Liste (Quelle „Excel“).

Zahlen werden direkt aus den Zellen gelesen; auch als Text gespeicherte deutsche Kommazahlen werden erkannt. Beim Layout »XY pairs« werden Zeilen mit leeren/ungültigen Zellen **pro Paar** verworfen — so lassen sich Spektren unterschiedlicher Länge in einem Blatt ablegen.

Voraussetzung: Der Excel-Import benötigt das Paket `openpyxl` (`pip install openpyxl`). Fehlt es, weist das Programm darauf hin.

Variante C — Excel-Ordner importieren

Wenn jedes Spektrum in einer **eigenen** Excel-Datei liegt, lässt sich ein ganzer Ordner auf einmal einlesen.

1. »**Excel folder ...**« klicken und den Ordner wählen. Alle enthaltenen `.xlsx` / `.xlsm` -Dateien werden gelesen (temporäre Excel-Sperrdateien wie `~$....` werden übersprungen).
2. Aus **jeder Datei** wird ein Spektrum gebildet (erste zwei Spalten = Energie, Intensität; weitere Spalten werden ignoriert). Die Dateien dürfen **verschiedene Energieraster** haben.
3. Im Dialog je Datei eine **Rolle** vergeben (Measured / Reference / Ignore). Enthält ein Dateiname „gemessen“/„measured“, wird diese Datei automatisch als **Measured** vorgeschlagen. Namen sind editierbar.
4. »**Apply**« — die Spektren landen in der Liste (Quelle „Excel“).

Variante D — Dateien laden

Für klassische zweispaltige Textdateien (Energie + Intensität):

- »**Load measured ...**« — eine Datei (das gemessene Spektrum).
- »**Load references ...**« — eine oder mehrere Dateien (die Komponenten).

Unterstützt werden `.dat`, `.csv`, `.txt`. Kommentarzeilen (beginnend mit `# ! % & * / ' "`) werden übersprungen; als Trennzeichen sind Leerzeichen, Komma, Tab und Semikolon erlaubt. Für **deutsche Dezimalkommas** vorher das Häkchen »**Decimal comma (files)**« setzen. Wie bei den XY-Paaren dürfen die Dateien unterschiedliche Raster haben — sie werden automatisch auf das gemeinsame Überlappungsfenster interpoliert.

Rollen nachträglich ändern

In der Liste eine Zeile auswählen und »**Switch role**« drücken, um zwischen *gemessen* und *Referenz* umzuschalten, oder »**Remove**«, um sie zu löschen. Mit **Doppelklick** (oder »**Rename**«) lässt sich eine Zeile **umbenennen**; Spektren ohne Namen werden automatisch nummeriert (»Reference 1«, »Reference 2«, ...). Zuletzt genutzter Ordner und Einstellungen werden gemerkt. Es ist immer nur **ein** gemessenes Spektrum aktiv; wird ein zweites als »gemessen« hinzugefügt, wird das bisherige automatisch zur Referenz.

4. Optionen (Abschnitt »2 - Options«)

Modus

Modus	Bedingung an die Gewichte	typische Verwendung
convex	$w_i \geq 0$ und $\sum_i w_i = 1$	Populationsanteile (Standard)
nnls	$w_i \geq 0$	nichtnegative Amplituden, ohne Summenbedingung
linear	keine	freie kleinste Quadrate; kann negative (unphysikalische) Gewichte liefern
all	—	rechnet convex/nnls/linear nebeneinander; der Plot zeigt <i>convex</i>

Hinweis zur Interpretation: Ein kleinerer R-Faktor bedeutet **nicht** automatisch das bessere Modell. *linear* hat die meisten Freiheitsgrade und erreicht fast immer den kleinsten R-Faktor — auch wenn die Gewichte physikalisch unsinnig sind. Für Populationsanteile ist *convex* die richtige Wahl.

Normierung

- **area** — Fläche = 1. Erst dadurch werden die *convex*-Gewichte zu echten Anteilen. **Empfohlen.**
- **max** — Maximum = 1.
- **none** — keine Normierung (Spektren werden so verwendet, wie sie sind).

Hinweis: *area* setzt eine **positive** Netto-Fläche voraus. Hat eine basislinienkorrigierte Kurve gleich viel negative wie positive Fläche (Netto-Fläche ≤ 0), meldet das Programm einen Fehler — dann *max/none* wählen oder das Fenster auf einen positiven Bereich einschränken.

Energiefenster

Energy min / max begrenzen den Fitbereich (in cm^{-1}). Beide leer lassen = automatische Überlappung aller Spektren. Nützlich, um z. B. nur den niederfrequenten Bereich (soft modes) anzupassen.

Δx -Gewichtung (ungleiches Raster)

Standard: **aus** (jeder Punkt zählt gleich — bit-identisch zur Kommandozeile). **An** gewichtet jeden Punkt mit seinem x-Abstand (Trapezregel); der Fit nähert dann das **Integral** von $(S_{\text{mess}} - S_{\text{fit}})^2$ über x an und hängt **nicht mehr von der Abtastdichte** ab. Sinnvoll bei **ungleichmäßigem** Raster (zusammengeführte Messungen, sehr unterschiedlich dicht abgetastete Referenzen); bei gleichmäßigem Raster ist der Effekt vernachlässigbar. R-Faktor, RMSE und R^2 werden dann als gewichtete (Integral-)Größen berechnet.

Bootstrap

Bootstrap N erzeugt Fehlerbalken der Gewichte (Standardabweichung und 95 %-Konfidenzintervall) per Residuen-Bootstrap — z. B. **500** . **0** schaltet ihn aus. **Seed** macht das Ergebnis reproduzierbar.

Der einfache Bootstrap zieht *einzelne* Residuen und nimmt an, dass sie **unabhängig** sind. Spektren-Residuen sind aber benachbart **korreliert**, weshalb diese Intervalle die Unsicherheit eher **unterschätzen**. Die Option »**Block bootstrap**« zieht stattdessen zusammenhängende Residuen-Blöcke ($\sim\sqrt{n}$), erhält die kurzreichweitige Korrelation und liefert ehrlichere (meist breitere) Intervalle. Bei *convex* sind Gewichte nahe 0 oder 1 an der Grenze gepinnt — dort ist die Verteilung einseitig, und der $\pm$ -Wert (Std) ist weniger aussagekräftig als das Perzentil-Intervall.

Komfort

Tooltips: Mauszeiger über »Mode«, »Normalization«, »Energy min« oder »Bootstrap« halten zeigt eine Kurzerklärung. **Tastenkürzel:** **F5** = Fit, **Strg+S** = Fit-Daten exportieren, **Strg+0** = Excel-Datei, **Strg+P** = Tabelle einfügen, **Esc** schließt Dialoge. Das zuletzt benutzte Verzeichnis wird gemerkt.

5. Ergebnis lesen (Abschnitt »3 - Result«)

Nach »**Run fit**« erscheinen:

- eine **Tabelle** mit Gewicht/Anteil je Referenz ($\pm$ -Wert und 95 %-Intervall bei aktivem Bootstrap, p-Wert aus dem F-Test), plus die Summe der Gewichte;
- die **Gütemaße**:
- **R-Faktor** =
$$\frac{\sum (S_{\text{mess}} - S_{\text{fit}})^2}{\sum S_{\text{mess}}^2}$$
 — kleiner ist besser (relativer Restfehler);
- **RMSE** — Wurzel des mittleren quadratischen Fehlers (absolut);
- **R²** — Bestimmtheitsmaß (1 = perfekt).

Ist der Bootstrap aktiv, werden Komponenten, deren 95 %-Intervall die **0 einschließt**, grau dargestellt und mit »~0« markiert — statistisch nicht gesichert, ggf. entbehrlich.

Die Spalte »**p (F)**« zeigt einen **F-Test** (drop-one): je Referenz wird der Fit *ohne* diese Komponente wiederholt und geprüft, ob das den Fit signifikant verschlechtert. **p < 0,05** $\Rightarrow$ die Komponente ist gerechtfertigt; größere Werte (graue Zeile) $\Rightarrow$ sie trägt nichts Gesichertes bei. **Wichtig:** Der F-Test setzt **unabhängige** Residuen voraus. Spektren-Residuen sind stark korreliert — das Ergebnis nennt die Lag-1-Korrelation ρ und die *effektive* Stichproben-größe N_{eff} . Die p-Werte sind daher **zu optimistisch** und nur als relativer Anhaltspunkt zu lesen; streng gilt der Test für *linear*, für *nnls/convex* ist er näherungsweise. Die Oberfläche zeigt **zusätzlich den mit N_{eff} korrigierten p-Wert**; beide p-Werte, ρ und N_{eff} stehen auch in den Exporten (.dat/JSON/Excel). Die belastbarere Unsicherheit ist das (Block-)Bootstrap-Intervall.

Der **Plot** zeigt oben das gemessene Spektrum, den Fit und die gewichteten Komponenten, unten das Residuum. Mit der **Werkzeugleiste** lässt sich zoomen, verschieben und der Plot direkt speichern. Mit den Feldern »**X axis**« und »**Y axis**« (Abschnitt »2 - Options«) lassen

sich die **Achsenbeschriftungen** frei setzen — z. B. »2 θ (°)« und »counts« für andere Datenarten; Standard ist »Energy (cm⁻¹)« bzw. »normalized intensity«. Das Feld »**x column name**« legt den Namen der **x-Spalte in den Exporten** (.dat/Excel) fest — Standard »energy«.

6. Interaktiver Overlay (Schaltfläche »Interactive overlay«)

Der interaktive Overlay öffnet ein eigenes Fenster, in dem sich jede Referenz über einen **Regler** stufenlos **nach oben/unten skalieren** lässt. Gezeigt werden das gemessene Spektrum, die **Summe** der gewichteten Referenzen ($\sum w_i \cdot S_i$) und die einzelnen Komponenten; darunter das Residuum. So lässt sich **mit dem Auge** ausprobieren, welche Mischung am besten passt.

Fensteraufteilung. Plot und Reglerbereich sind durch eine **ziehbare Trennleiste** getrennt. Ziehen Sie die Leiste nach **unten**, um dem Reglerbereich mehr Platz zu geben (z. B. bei vielen Referenzen oder einem niedrigen Fenster), oder nach **oben** für einen größeren Plot. Das Fenster ist frei skalierbar; beim Öffnen ist der gesamte Reglerbereich sichtbar.

Wichtig: Der Overlay dient der **Exploration**. Die zu berichtenden Zahlen stammen aus dem automatischen Fit (mit Bootstrap-Konfidenzintervallen).

Live-Anzeigen (aktualisieren bei jedem Reglerzug):

- der **Anteil** jeder Komponente am Gesamtmodell ($w_i / \sum w_j$, in %),
- **R-Faktor** und **R²** der aktuellen Summe.

Reglerskalierung. Jeder Regler läuft von 0 bis zu einem **datenabhängigen Maximum** — dem Doppelten des größten der folgenden Werte: Auto-Fit-Gewicht, „Flächen-Match" (die Komponente trägt allein die gesamte Messfläche) und „Peak-Match". Der Flächen-Match als Untergrenze hält auch einen Regler nutzbar, den der Fit auf ~ 0 gesetzt hat. Dadurch sind die Regler **sinnvoll skaliert**, unabhängig von der Normierung: bei **area** liegen die Werte im gut lesbaren Bereich 0 ... ≈ 2 ; bei **none** passen sich die Maxima an die Rohintensitäten an (entsprechend große Zahlen).

Neben jedem Regler zeigt ein **Zahlenfeld** den exakten Wert und erlaubt die direkte Eingabe (auch mit Dezimalkomma). Ein Wert über das aktuelle Maximum hinaus **erweitert die Reglerspanne automatisch** — beliebiges „Übertreiben" bleibt also möglich, ohne dass der Regler im Normalbereich unbrauchbar wird.

Knöpfe:

- »**Load from auto-fit**« — setzt die Regler auf die *convex*-Gewichte des letzten Fits.
- »**Reset**« — setzt alle Regler auf 0.

Export aus dem Overlay (gleiche Formate wie der Haupt-Export):

- »**Save image**« — die Abbildung (.png / .pdf / .svg),
- »**Data (.dat/.csv)**« — Energie, measured, fit, residual, gewichtete Komponenten,
- »**Excel (.xlsx)**« — Blätter »fit« und »summary«,
- »**JSON**« — Gewichte, Summen und Gütemaße.

Hinweis: Der Overlay verwendet dieselben aufbereiteten Daten wie der Fit (Energiefenster und Normierung gelten also weiter). Nach Änderung von Fenster oder Normierung den Fit neu ausführen und den Overlay erneut öffnen.

7. Export (Abschnitt »4 - Export«)

Schaltfläche	Format	Inhalt
Fit data (.dat/.csv)	Tab-getrennt	Energie, measured, fit, residual, gewichtete Komponenten — direkt in Origin importierbar
Fit data (.dat/.csv)	Komma-getrennt	wie oben
Excel (.xlsx)	Arbeitsmappe	Blatt »fit« (die Spalten) + Blatt »summary« (Gewichte & Gütemaße) — benötigt <code>pandas</code> + <code>openpyxl</code>
Summary (.json)	JSON	alle Gewichte, Summen, Gütemaße (maschinenlesbar)
Plot (.png)	.png / .pdf / .svg	die Abbildung

Die `.dat` / `.csv`-Dateien tragen einen **Kommentar-Kopf** (Zeilen mit `#`) mit Programm/Version, Modus, Normierung, Fitfenster und den Gewichten — zur Reproduzierbarkeit. Origin & Co. überspringen Kommentarzeilen automatisch.

Zusätzlich kopiert »**Copy results**« (Abschnitt »3 - Result«) die Ergebnis-tabelle samt Gütemaßen als Text (Tab-getrennt) in die Zwischenablage.

8. Typische Probleme

- **»Empty fit window«** — die Energiebereiche der Spektren überlappen nicht, oder Emin/Emax sind zu eng gesetzt. Grenzen prüfen oder leeren.
- **Falsch eingelesene Zahlen** (z. B. um Faktor 1000 daneben oder „NaN“) — Trennzeichen/Dezimaltrennzeichen im Einfüge-Dialog explizit setzen. Häufige Ursache: deutsches Komma wurde als Spaltentrenner missdeutet oder umgekehrt.
- **»exactly one measured spectrum«** — in der Liste die Rollen prüfen; ggf. mit »Switch role« korrigieren.
- **Negative Gewichte** — Modus ist *linear*. Für physikalische Anteile auf *convex* (oder *nnls*) wechseln.
- **.xlsx-Export ohne Wirkung** — `pandas` / `openpyxl` nicht installiert (siehe Abschnitt 2). Die anderen Exportformate funktionieren trotzdem.
- **»area normalization not meaningful«** — die Netto-Fläche ist ≤ 0 (gleich viel negative wie positive Fläche). *max/none* wählen oder das Fenster auf einen positiven Bereich einschränken.
- **Excel-Zellen nach Import leer** — Formelzellen brauchen gespeicherte Werte; die Datei einmal in Excel öffnen und speichern.

9. Schnelltest mit Beispieldaten

Im Lieferumfang liegt `beispiel_tabelle.txt` — eine fertige Tabelle (tab-getrennt, deutsche Dezimalkommas, Kopfzeile `Energie Gemessen WT DM Red`). Damit lässt sich Variante A sofort ausprobieren:

1. Datei in einem Texteditor öffnen, **alles markieren (Strg + A), kopieren**.
2. Im Programm »**Paste table ...**« → einfügen → »**Detect columns**«.
3. Rollen prüfen (Energy / Measured / 3× Reference) → »**Apply**«.
4. Modus *convex*, Normierung *area*, »**Run fit**«.

Erwartetes Ergebnis: Anteile $\approx$ **WT 0,55 · DM 0,30 · Red 0,15** bei einem R-Faktor um $1,4 \cdot 10^{-3}$ — die in die Daten eingebauten „wahren“ Werte.

10. Hinweis: Kommandozeile

Für **Stapelverarbeitung und Skripte** gibt es die Kommandozeile `pylcf-cli` (bzw. `python -m pylcf.cli`). GUI und CLI nutzen denselben Kern (`pylcf/core.py`) und liefern damit dieselben Gewichte und Gütemaße. Die CLI schreibt `.dat` und `.json`, auf Wunsch (`--xlsx`) auch die Excel-Mappe, `--csv` schreibt kommagetrennt statt Tab, `--quiet` unterdrückt die Ausgabe, und `--xlabel` / `--ylabel` setzen die Plot-Achsen; `pylcf-cli --help` listet alle Optionen.

Eine Kurzfassung dieser Anleitung ist im Programm selbst über »**Show guide**« abrufbar.

11. Referenzen

Die im Programm genutzten Methoden und der Software-Stack:

- **NNLS** (nicht-negative kleinste Quadrate): C. L. Lawson, R. J. Hanson, *Solving Least Squares Problems*, SIAM (1995).
- **SLSQP** (für den *convex*-Fit): D. Kraft, *A Software Package for Sequential Quadratic Programming*, DLR-FB 88-28 (1988).
- **Bootstrap**: B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall (1993).
- **Verwandte LCF-Software (XAS)**: B. Ravel, M. Newville, *ATHENA, ARTEMIS, HEPHAESTUS: data analysis for X-ray absorption spectroscopy using IFEFFIT*, J. Synchrotron Rad. **12**, 537–541 (2005); M. Newville, *Larch: An Analysis Package for XAFS and Related Spectroscopies*, J. Phys. Conf. Ser. **430**, 012007 (2013).
- **Software-Stack**: C. R. Harris et al., *Array programming with NumPy*, Nature **585**, 357–362 (2020); P. Virtanen et al., *SciPy 1.0*, Nat. Methods **17**, 261–272 (2020); J. D. Hunter, *Matplotlib: A 2D Graphics Environment*, Comput. Sci. Eng. **9**(3), 90–95 (2007).